

特 征 工 程

2025年5月

TRANSWARP
星 环 科 技



目录> CONTENTS

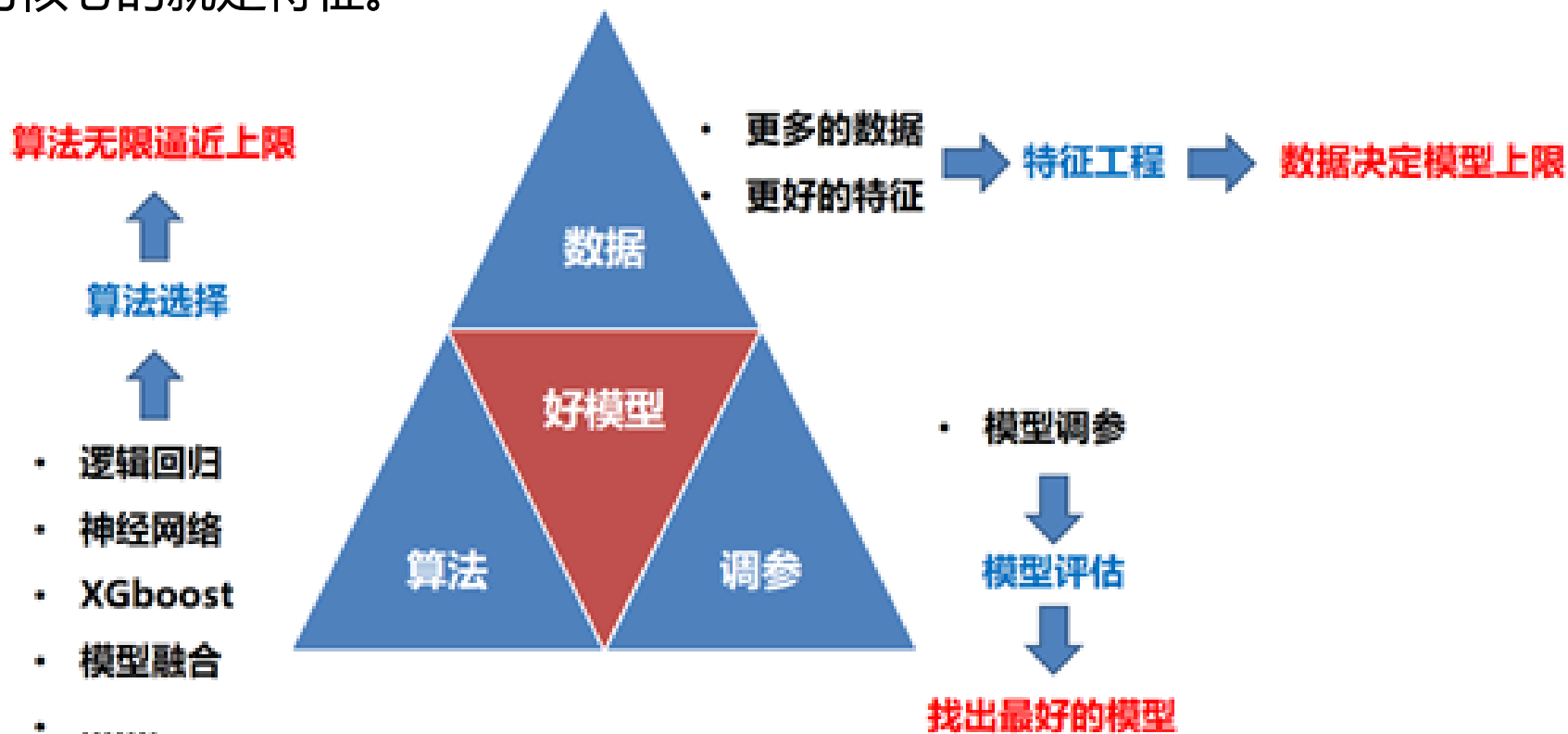
- 1 特征工程概述
- 2 特征提取
- 3 特征组合
- 4 GBDT特征提取

1
chapter

特征工程概述

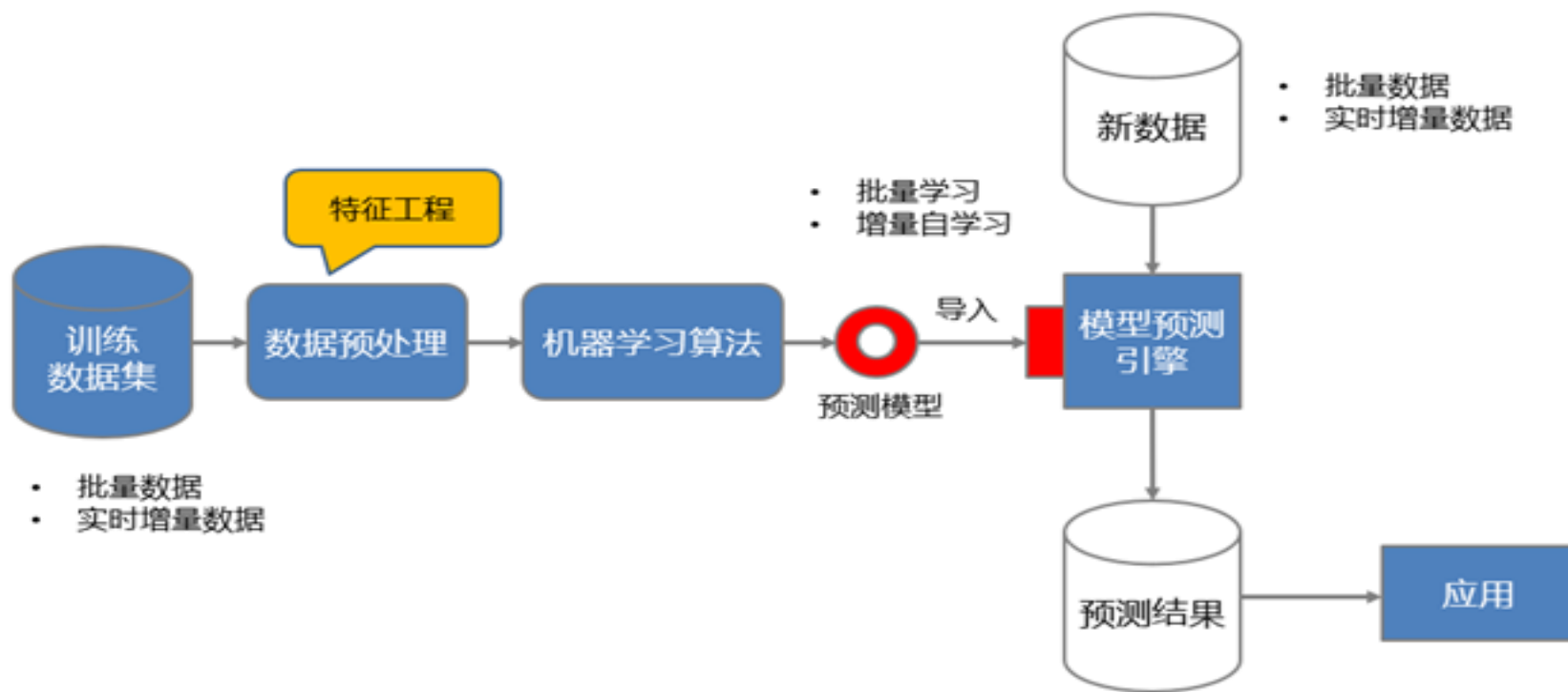
什么是特征工程

- 影响模型的三要素，其中非常重要的一个因素就是数据。
- 我们经常说的一句话叫“**数据决定了模型的上限，算法只是无限逼近这个上限**”。
- 数据的核心就是特征。



什么是特征工程

- 一切围绕着特征展开的工作我们都可以称为“特征工程”，吴恩达有一句关于特征工程的话道出了应用机器学习的真相。“**应用机器学习其实就是在做特征工程，特征工程是非常难、耗时、也是需要专业知识的一个工作。**”——吴恩达
- 特征工程也可以理解为机器学习模型开发流程中的数据预处理。



特征工程主要包括如下工作：

- 特征提取
- 特征转换
- 特征选择
- 特征降维

特征提取

使用各种方法尽最大可能地从原始数据中提取出有价值的特征变量供模型所用，常用的特征提取方法有：

- **行为特征提取**：如何从交易型数据中提取用户的行为特征，典型的就是 **RFM** 指标（此部分内容将来单独提出来讲）。
- **文本特征提取**：如何对一段文字或者一篇文档进行特征提取，用于文档分类和聚类等，典型的方法是 TFIDF（此部分内容将来单独提出来讲）
- **图像特征提取**：如何提取一张图片的特征用于图像识别和搜索等，典型的特征有HOG, LBP, Haar, 还有基于深度神经网络提取的深度特征（单独讲）
- **基于已有特征进行组合得到新的特征**：例如构造 x_1x_2 这样的组合特征（ x_1 和 x_2 是两个独立的特征）
- **特征降维**：对大量的原始特征进行降维，提取数量较少的主成分作为新的特征，达到减少特征变量数量的目的。
- **基于模型的特征提取**：例如决策树模型中，每一条样本所属的叶子节点编号就是一个非常重要的特征。

特征转换

特征转换就是对原始的特征变量进行转换处理，主要的原因是算法的需要，常做的特征转换有：

- **标准化处理**：把特征变量转换为均值为0，方差为1
- **归一化处理**：把特征变量转换为最小值为0，最大值为1
- **正则化处理**：将每个样本在所有变量上的值缩放到单位范数（即每个样本在所有变量上的值的范数为1）
- **特征编码**：例如常用的独热编码

特征选择

经过了特征提取和特征转换之后，我们会得到若干个特征变量，一种方法是全部喂给算法让算法自己来决定用哪个，另外一种方法是先对特征进行筛选，然后再喂给算法，这个工作就叫特征选择。特征选择也有很多方法：

- **过滤法**：根据特征变量的统计指标来筛选，例如根据特征变量的方差，特征与目标变量的相关系数来筛选特征
- **递归特征消除法**：基于某一个模型，每一次训练都移除掉一个最不重要的特征，直到满足最大特征数要求就停止
- **模型筛选法**：基于模型来筛选变量，有一些模型会对计算变量的重要性，例如决策树，随机森林，GBDT等树模型

特征降维

特征降维也是将数据的特征个数减少，跟特征选择不同的，特征降维保留的特征不再是原有特征。主要方法有：

- **主成分分析 PCA**：无监督降维
- **线性判别分析 LDA**：有监督降维

为什么要做特征工程

- **特征越好，模型灵活性越强**——可以选择更多的算法来训练模型
- **特征越好，模型越简单**——简单的模型不容易过拟合，训练和预测速度更快
- **特征越好，模型效果越出色**——特征决定了模型性能的上限



2 chapter

特征提取

- ✓ 日期时间特征提取
- ✓ 行为特征提取
- ✓ 序列特征提取
- ✓ 文本特征提取

日期型变量

日期型变量是一种比较特殊的变量，它既不能当做数值变量（类似年龄），也不能当做类别变量（类似性别）来处理。我们需要对日期变量进行一些处理，从中提取有价值的信息后，才能用于后续分析和建模。

日期型变量又包含两种类型：

- 只包含日期，例如2018-01-01
- 同时包含日期和时间，例如2018-01-01 08:30:25

日期时间特征

从一个日期数据 (例如2018-01-01) 中能提取哪些信息呢?

- 日期的年份: 2018年
 - 日期的月份: 1月
 - 日期的号数: 1号
 - 日期所在的季度: 1季度
 - 一年中的上半年还是上半年: 上半年
 - 一年中的第几周: 第1周
 - 一周中的第几天: 第2天 (星期一)
 - 周末还是非周末: 非周末
 - 是否节假日: 元旦节
 - 一年中的第几天: 第1天
 - 离年末还剩多少天: 364天
 - 距离今天的天数: 假如今天是 2018-01-15, 那么就是14天
- 如果日期数据中包含了时间, 例如 2018-01-01 08:30:25, 我们还可以提取:
- 日期中的小时数: 8
 - 日期中的分钟数: 30
 - 日期中的秒数: 25

日期时间特征提取案例

导入包和数据

```
# 导入相关包
import pandas as pd
import datetime as dt
import numpy as np
# 导入数据
df = pd.read_csv('./datasets/Retail_Data_Transactions.csv')
# 我们注意到, 使用默认的pandas.read_csv()导入数据, trans_date并没有被识别为日期型变量,
# 我们需要转换一下, 把trans_date转换为日期型变量, 重新赋予一个新的变量
df['date'] = pd.to_datetime(df['trans_date'])
df.head(5)
```

	customer_id	trans_date	tran_amount	date
0	CS4441	24/1/2015	984	2015-01-24
1	CS5472	18/5/2014	206	2014-05-18
2	CS7781	1/12/2013	466	2013-01-12
3	CS7717	18/12/2015	782	2015-12-18
4	CS8746	30/12/2012	372	2012-12-30

日期时间特征提取案例

提取特征

```
import datetime
df['year'] = df['date'].dt.year #年
df['month'] = df['date'].dt.month #月
df['day'] = df['date'].dt.day #日
df['quarter'] = df['date'].dt.quarter #季度
df['week'] = df['date'].dt.week #一年的第几周
df['weekday'] = df['date'].dt.weekday #一周的第几天
df['halfyear'] = df['date'].map(lambda d: 'H1' if d.month<=6 else 'H2') #上下半年
df['dayofyear'] = df['date'].dt.dayofyear #一年中的第几天
#距离今天的天数（就是RFM中的R）
today = datetime.date(2015,4,1) #定义今天的日期
df['daysToToday'] = (today - df['date'].dt.date).dt.days
df.head()
```

	customer_id	trans_date	tran_amount	date	year	month	day	quarter	week	weekday	halfyear	dayofyear	daysToToday
0	CS4441	24/1/2015	984	2015-01-24	2015	1	24	1	4	5	H1	24	67
1	CS5472	18/5/2014	206	2014-05-18	2014	5	18	2	20	6	H1	138	318
2	CS7781	1/12/2013	466	2013-01-12	2013	1	12	1	2	5	H1	12	809
3	CS7717	18/12/2015	782	2015-12-18	2015	12	18	4	51	4	H2	352	-261
4	CS8746	30/12/2012	372	2012-12-30	2012	12	30	4	52	6	H2	365	822

日期时间特征提取案例

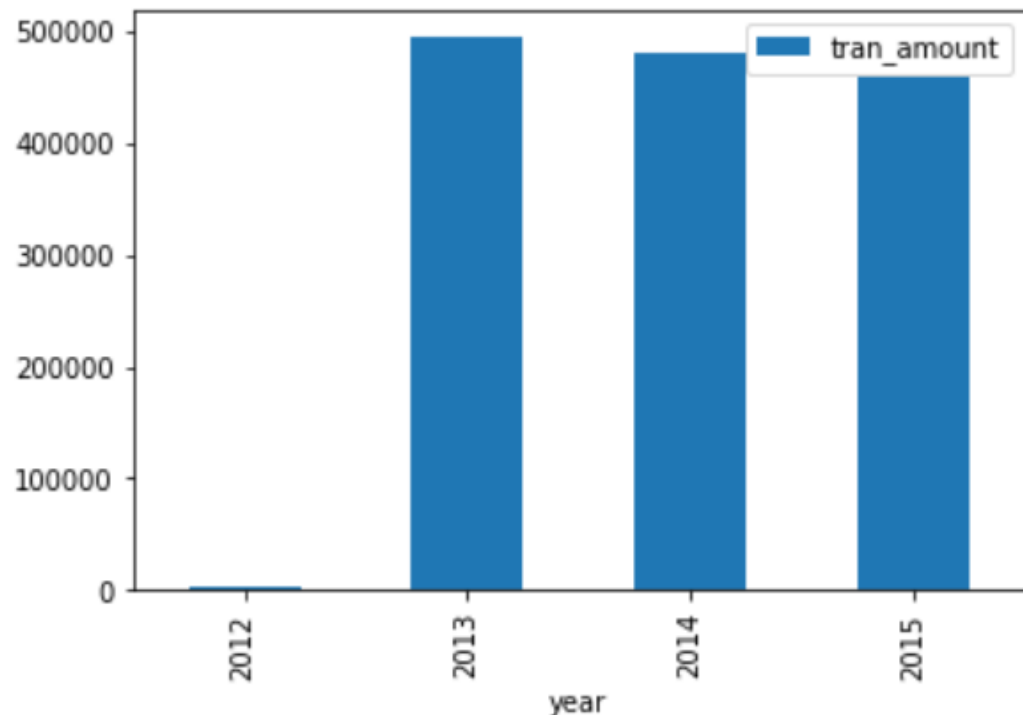
消费金额分析

按年对消费金额进行分析

```
amount_byyear = df.groupby('year').agg({'tran_amount': 'sum'})
```

```
amount_byyear
```

```
amount_byyear.plot(kind='bar')
```

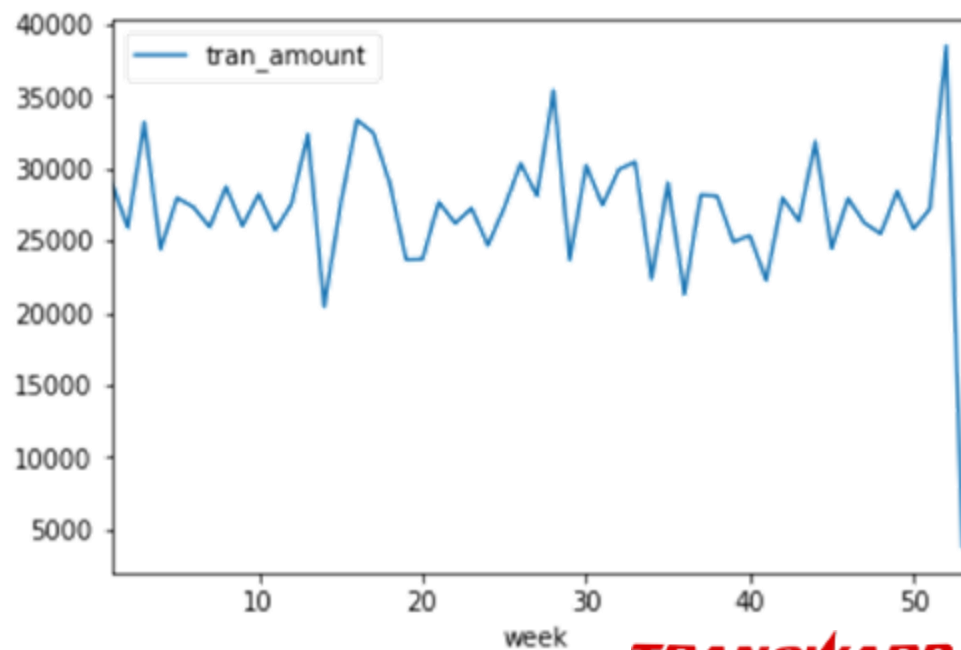


按week (一年的第几周) 对消费金额进行分析

```
amount_byweek = df.groupby('week').agg({'tran_amount': 'sum'})
```

```
amount_byweek.tail()
```

tran_amount	
week	
49	28441
50	25804
51	27205
52	38552
53	3714



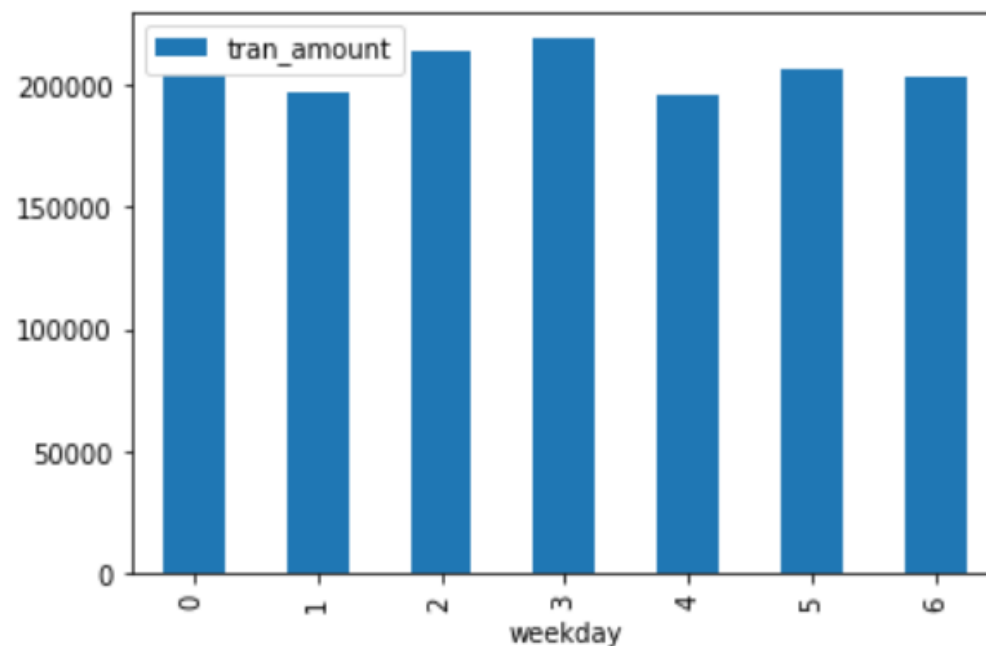
日期时间特征提取案例

消费金额分析

```
# 按weekday（一周的第几天）对消费金额进行分析  
amount_byweekday = df.groupby('weekday').agg({'tran_amount': 'sum'})  
amount_byweekday
```

tran_amount	
weekday	
0	203915
1	196745
2	213452
3	218419
4	195415
5	206070
6	202695

```
amount_byweekday.plot(kind='bar')
```



RFM 特征提取

RFM 是 **CRM**（客户关系管理）领域中广泛使用的一个评估用户价值的指标。**CRM** 客户数据库中有 3 个神奇的要素，这 3 个要素构成了 **CRM** 数据分析最好的指标：

- 最近一次购买时间 (Recency)
- 购买频率 (Frequency)
- 购买金额 (Monetary)

左边是用户的消费行为数据，它包含了三个字段：用户ID，消费时间，消费金额。右边是基于用户消费行为数据提取出来的三个特征：R，F，M。

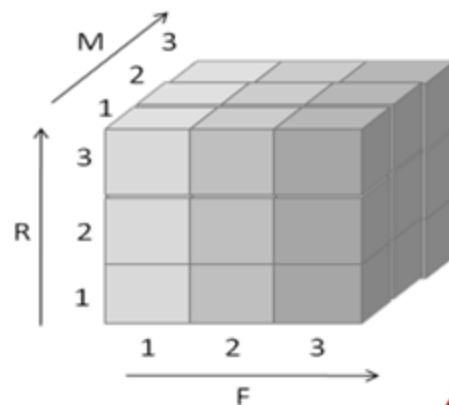
用户交易数据：用户ID，时间，金额

CardID,	Date,	Amount↓
"C0100000199",	2011/08/20,	229.000000↓
"C0100000199",	2011/06/28,	139.000000↓
"C0100000199",	2011/12/29,	229.000000↓
"C0100000343",	2011/07/27,	49.000000↓
"C0100000343",	2011/02/02,	169.990000↓
"C0100000343",	2011/07/12,	299.000000↓
"C0100000343",	2011/02/02,	34.950000↓
"C0100000343",	2011/09/07,	99.000000↓
"C0100000343",	2011/05/13,	49.000000↓
"C0100000375",	2011/09/22,	99.990000↓
"C0100000375",	2011/05/02,	5.990000↓
"C0100000375",	2011/11/01,	49.000000↓
"C0100000375",	2011/10/16,	69.000000↓
"C0100000482",	2011/08/12,	84.000000↓
"C0100000482",	2011/03/28,	69.000000↓
"C0100000482",	2011/04/03,	24.990000↓
"C0100000482",	2011/12/10,	19.990000↓
"C0100000689",	2011/05/23,	79.000000↓



RFM:

- 最近一次消费时间(**R**ecency)
- 消费频率(**F**requency)
- 购买金额(**M**onetary)



RFM 特征提取

RFM 其实并不是只适用于 CRM 领域。我们把上图左边的这种格式的数据叫做“**交易数据** (Transactional Data)”，在很多行业，这样的交易数据很常见，它们都可以使用 RFM 的方法来提取特征，例如：

- 店铺的用户购买订单：**用户ID，下单时间，下单频率，下单金额**
- 银行的客户交易流水：**用户ID，交易时间，交易频率，交易金额**
- 用户的上网日志：**用户ID，登陆时间，登陆频率，停留时长**
- 用户使用APP的日志：**用户ID，打开时间，打开频率，使用时长**

所以我们看到：**RFM** 就是用来衡量**用户行为特征**最重要的一种方法。

RFM 原始特征提取

提取用户的 RFM 的特征过程很简单：针对 **用户ID** 进行汇总 (GroupBy)

- Recency 就是用户消费日期的最大值Max(Date)，一般我们还会用当前日期减去它转换为距离现在的间隔时长 (天数)
- Frequency 就是用户的消费记录数Count(*)
- Monetary 就是用户消费额的总和Sum(Amount)

CardID.	Date.	Amount↓
"C0100000199",	2011/08/20,	229.000000↓
"C0100000199",	2011/06/28,	139.000000↓
"C0100000199",	2011/12/29,	229.000000↓
"C0100000343",	2011/07/27,	49.000000↓
"C0100000343",	2011/02/02,	169.990000↓
"C0100000343",	2011/07/12,	299.000000↓
"C0100000343",	2011/02/02,	34.950000↓
"C0100000343",	2011/09/07,	99.000000↓
"C0100000343",	2011/05/13,	49.000000↓
"C0100000375",	2011/09/22,	99.990000↓
"C0100000375",	2011/05/02,	5.990000↓
"C0100000375",	2011/11/01,	49.000000↓
"C0100000375",	2011/10/16,	69.000000↓
"C0100000482",	2011/08/12,	84.000000↓
"C0100000482",	2011/03/28,	69.000000↓
"C0100000482",	2011/04/03,	24.990000↓
"C0100000482",	2011/12/10,	19.990000↓
"C0100000689",	2011/05/23,	79.000000↓



CardID	Recency	Frequency	Monetary
C0100000199	2011/12/29	3	597

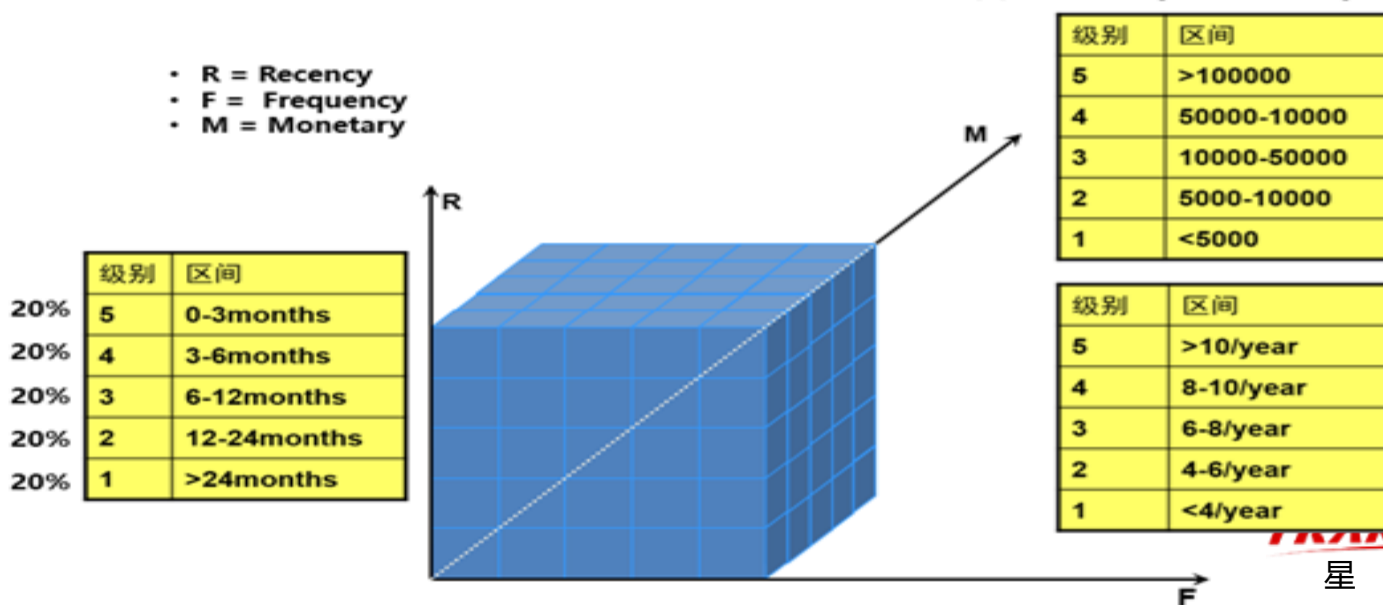
Group by CardID:

- Recency = **now**() - **max**(Date)
- Frequency = **count** (*)
- Monetary = **sum** (Amount)

RFM 特征离散化

原始提取出来的 **RFM特征** 都是 **连续型** 的数值，RFM分析的时候我们习惯把它们进行 **离散化**（前面我们有学习过，又叫**分箱**），常见的离散化方法是使用之前介绍过的等深分段的方法，把 **RFM** 切分为人数相同的5个等分，即**R取值{1,2,3,4,5}**，**F取值{1,2,3,4,5}**，**M取值{1,2,3,4,5}**，这样把RFM组合在一起可以得到一个新的特征取值是{111,112,113...551,552,553,554,555}共计 **$5*5*5=125$** 个值，这又叫RFM立方体，125个值就是立方体中的125个块。

- 把RFM分别切分为5个人数等分区间，然后形成RFM的组合立方体（111~555）



RFM 特征提取案例

导入数据

```
import time # 时间库
import numpy as np # numpy库
import pandas as pd # pandas库
from pyecharts.charts import Bar3D # 3D柱形图
#导入数据
sheet_names = ['2015', '2016', '2017', '2018', '会员等级']
sheet_datas = [pd.read_excel('data/sales.xlsx', sheet_name=name) for name in sheet_names]
```

#查看数据基本情况

```
for each_name, each_data in zip(sheet_names, sheet_datas):
    print('[data summary for ====={}=====]'.format(each_name))
    print('Overview:', '\n', each_data.head(4)) # 展示数据前4条
    print('DESC:', '\n', each_data.describe()) # 数据描述性信息
    print('NA records', each_data.isnull().any(axis=1).sum()) # 缺失值记录数
    print('Dtypes', each_data.dtypes) # 数据类型
```

[data summary for =====2015=====]

Overview:

	会员ID	订单号	提交日期	订单金额
0	1.527800e+10	3.000305e+09	2015-01-01	499.0
1	3.923638e+10	3.000306e+09	2015-01-01	2588.0
2	3.872204e+10	3.000642e+09	2015-01-01	498.0
3	1.104964e+10	3.000799e+09	2015-01-01	1572.0

DESC:

	会员ID	订单号	订单金额
count	3.077400e+04	3.077400e+04	30774.000000
mean	2.918779e+10	4.020414e+09	960.991161
std	1.385333e+10	2.630510e+08	2068.107231
min	2.670000e+02	3.000305e+09	0.500000
25%	1.944122e+10	3.885510e+09	59.000000
50%	3.746545e+10	4.117491e+09	139.000000
75%	3.923593e+10	4.234882e+09	899.000000
max	3.954613e+10	4.282025e+09	111750.000000

NA records 30596

Dtypes 会员ID float64

RFM 特征提取案例

数据预处理

```
# 去除缺失值和异常值
for ind, each_data in enumerate(sheet_datas[:-1]):
    sheet_datas[ind] = each_data.dropna() # 丢弃缺失值记录
    sheet_datas[ind] = each_data[each_data['订单金额'] > 1] # 丢弃订单金额<=1的记录
    sheet_datas[ind]['max_year_date'] = each_data['提交日期'].max() # 增加一列最大日期值
```

RFM 特征提取案例

合并4个年份的数据

```
# 汇总所有数据
data_merge = pd.concat(sheet_datas[:-1], axis=0)
# 获取各自年份数据
data_merge['date_interval'] = data_merge['max_year_date'] - data_merge['提交日期']
data_merge['year'] = data_merge['提交日期'].dt.year
# 转换日期间隔为数字
data_merge['date_interval'] = data_merge['date_interval'].apply(lambda x: x.days)
data_merge.head()
```

	会员ID	订单号	提交日期	订单金额	max_year_date	date_interval	year
0	1.527800e+10	3.000305e+09	2015-01-01	499.0	2015-12-31	364	2015
1	3.923638e+10	3.000306e+09	2015-01-01	2588.0	2015-12-31	364	2015
2	3.872204e+10	3.000642e+09	2015-01-01	498.0	2015-12-31	364	2015
3	1.104964e+10	3.000799e+09	2015-01-01	1572.0	2015-12-31	364	2015
4	3.503875e+10	3.000822e+09	2015-01-01	10.1	2015-12-31	364	2015

RFM 特征提取案例

提取原始rfm特征

```
# 按会员ID做汇总
rfm_gb = data_merge.groupby(['year', '会员ID'], as_index=False).agg(
    {'date_interval': 'min', # 计算最近一次订单时间
     '提交日期': 'count', # 计算订单频率
     '订单金额': 'sum'}) # 计算订单总金额
# 重命名列名
rfm_gb.columns = ['year', '会员ID', 'r', 'f', 'm']
rfm_gb.head()
```

	year	会员ID	r	f	m
0	2015	267.0	197	2	105.0
1	2015	282.0	251	1	29.7
2	2015	283.0	340	1	5398.0
3	2015	343.0	300	1	118.0
4	2015	525.0	37	3	213.0

RFM 特征提取案例

分箱

查看数据分布

```
desc_pd = rfm_gb.iloc[:, 2:].describe().T  
desc_pd
```

	count	mean	std	min	25%	50%	75%	max
r	148591.0	165.524043	101.988472	0.0	79.0	156.0	255.0	365.0
f	148591.0	1.365002	2.626953	1.0	1.0	1.0	1.0	130.0
m	148591.0	1323.741329	3753.906883	1.5	69.0	189.0	1199.0	206251.8

定义区间边界

```
r_bins = [-1, 79, 255, 365] # 注意起始边界小于最小值  
f_bins = [0, 2, 5, 130]  
m_bins = [0, 69, 1199, 206252]
```

RFM 特征提取案例

分箱

```
# RFM分箱得分
rfm_gb['r_score'] = pd.cut(rfm_gb['r'], r_bins, labels=[i for i in range(len(r_bins)-1,0,-1)]) # 计算R得分
rfm_gb['f_score'] = pd.cut(rfm_gb['f'], f_bins, labels=[i+1 for i in range(len(f_bins)-1)]) # 计算F得分
rfm_gb['m_score'] = pd.cut(rfm_gb['m'], m_bins, labels=[i+1 for i in range(len(m_bins)-1)]) # 计算M得分
#计算RFM组合
rfm_gb['r_score'] = rfm_gb['r_score'].astype(np.str)
rfm_gb['f_score'] = rfm_gb['f_score'].astype(np.str)
rfm_gb['m_score'] = rfm_gb['m_score'].astype(np.str)
rfm_gb['rfm_group'] = rfm_gb['r_score'].str.cat(rfm_gb['f_score']).str.cat(rfm_gb['m_score'])
rfm_gb.head()
```

	year	会员ID	r	f	m	r_score	f_score	m_score	rfm_group
0	2015	267.0	197	2	105.0	2	1	2	212
1	2015	282.0	251	1	29.7	2	1	1	211
2	2015	283.0	340	1	5398.0	1	1	3	113
3	2015	343.0	300	1	118.0	1	1	2	112
4	2015	525.0	37	3	213.0	3	2	2	322

RFM 特征提取案例

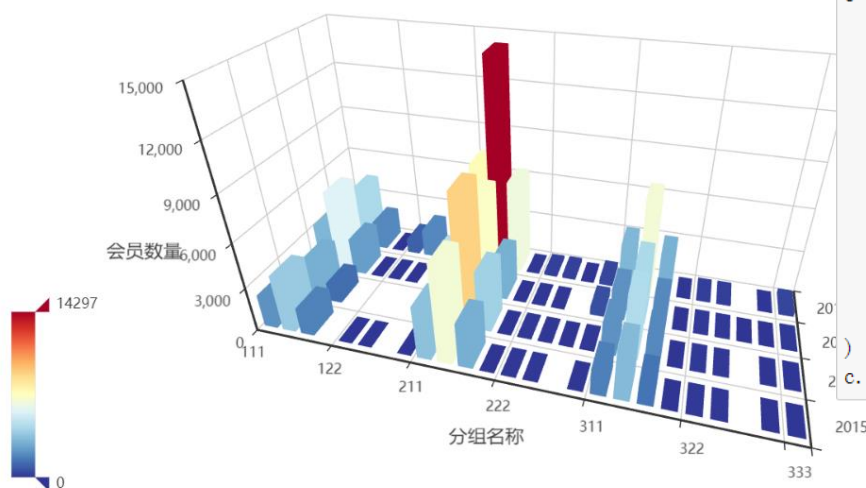
3D可视化

图形数据汇总

```
display_data = rfm_gb.groupby(['rfm_group', 'year'], as_index=False)['会员ID'].count()
display_data.columns = ['rfm_group', 'year', 'number']
display_data['rfm_group'] = display_data['rfm_group'].astype(np.int32)
display_data.head()
```

	rfm_group	year	number
0	111	2015	2180
1	111	2016	1498
2	111	2017	3169
3	111	2018	2271
4	112	2015	3811

RFM分组结果



显示图形

```
from pyecharts.commons.utils import JsCode
from pyecharts import options as opts
range_color = ['#313695', '#4575b4', '#74add1', '#abd9e9', '#e0f3f8', '#ffffbf',
               '#fee090', '#fdae61', '#f46d43', '#d73027', '#a50026']
range_max = int(display_data['number'].max())
c = (
    Bar3D() #设置了一个3D柱形图对象
    .add(
        "", #标题
        [d.tolist() for d in display_data.values], #数据
        xaxis3d_opts=opts.Axis3DOpts(type_='category', name='分组名称'), #x轴数据类型, 名称
        yaxis3d_opts=opts.Axis3DOpts(type_='category', name='年份'), #y轴数据类型, 名称
        zaxis3d_opts=opts.Axis3DOpts(type_='value', name='会员数量'), #z轴数据类型, 名称
    )
    .set_global_opts(#设置颜色, 及不同取值对应的颜色
        visualmap_opts=opts.VisualMapOpts(max_=range_max, range_color=range_color),
        title_opts=opts.TitleOpts(title="RFM分组结果"), #设置标题
    )
    .render_notebook() #在notebook中显示
```

请用一句SQL取出所有用户对商品的行为特征，特征分为已购买、购买未收藏、收藏未购买、收藏且购买（输出结果如下表）

id	user_id	item_id	pay_time	item_num
1	001	201	2018-08-31 00:00:01	1
2	002	203	2018-09-02 12:00:02	2
3	003	203	2018-09-01 00:00:01	1
4	003	203	2018-09-04 09:10:30	1

id	user_id	item_id	fav_time
1	001	201	2018-08-31 00:00:01
2	002	202	2018-09-02 12:00:02
3	003	204	2018-09-01 00:00:01

user_id	item_id	已购买	购买未收藏	收藏未购买	收藏且购买
001	201	1	0	0	1
002	202	0	0	1	0
002	203	1	0	0	0
003	203	1	0	0	0
003	204	0	0	1	0

时间序列数据

时间序列数据主要有两种类型：

- 经典时间序列
- 带时间戳的事件流

经典时间序列

经典时间序列数据包含随时间测量的一系列数值，这些时间点分布是均匀的，例如每小时，每天，每周，每月等，下面是一些典型的例子：

- 股票每天的收盘价
- 每天的最高气温和最低气温
- 某一个城市每一天的电力消耗量
- 病人每一天的血压值
-

带时间戳的事件流

带时间戳的事件流数据包含了离散的一系列事件的时间戳，以及对应的事件相关的信息，典型的例子有：

- 用户访问网站的点击事件流，记录了用户每次点击的时间以及网页URL
 - 用户银行账户的交易记录，包含了每一次交易的时间，类型（存款，取款）以及金额
 - 用户在店铺的历史购买记录，包含了每一次购买的时间，商品ID，数量，金额等
- 一般使用 RFM 的方式处理

时间序列建模

时间序列数据最重要的建模应用就是预测，对时间序列的预测有两种方法：

- **时间序列预测**：基于过去的序列值对未来的序列值进行预测，例如：
 - 预测明天某一只股票的价格
 - 预测明天的天气
 - 预测明年的汽车销量
- **时间序列分类/回归**：它与时间序列预测的主要区别是：时间序列分类/回归是对成百上千的时间序列（你可以理解为机器学习中的训练样本）进行分类或者回归，而不是预测单个时间序列的数值。例如：如果我们根据某一只股票的历史价格对未来几天的价格进行预测这就是时间序列预测

时间序列特征提取方法

因为时间序列是通过历史来预测未来，那么，这个时序值的历史数据，也就是当前时间点之前的信息就非常有用，通过他可以发现时间序列的趋势因素、季节性周期性因素以及一些不规则的变动，具体来说这部分特征可以分为两种：

- 滞后值
- 滑动窗口统计

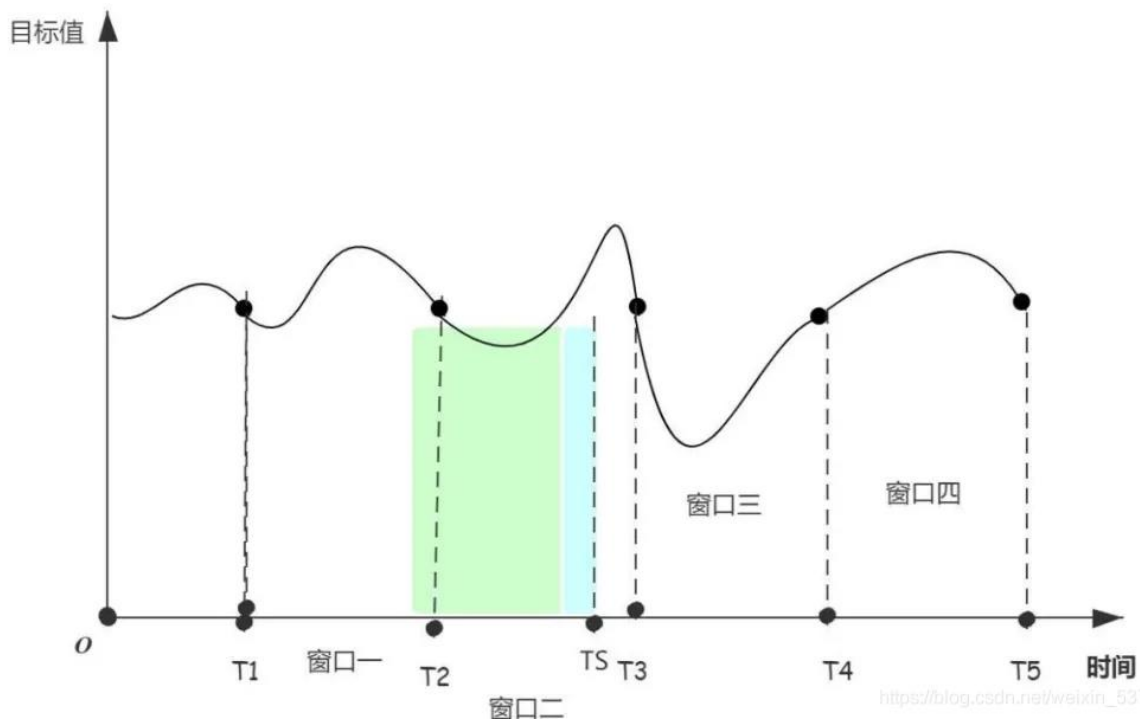
滞后值

- 也称 **lag feature**，比如对于 t 时刻的数据，我们认为它是跟**昨天**的数据、**上周同一天**的数据、**上个月同一天**的数据、**去年同期**的数据是高度相关的，那么，我们就可以将 $t-1$ 、 $t-7$ 、 $t-30$ 、 $t-365$ 的数据用来做特征。
- 注意：如果预测的 horizon 超过了滞后的期数，那么这时候就得使用递归的方式，将先前预测的值作为特征

滑动窗口统计

滑动窗口统计 (Rolling Window Statistics) , 使用先前时间观察值的统计信息作为特征

- window=7: 对于 t 时刻, 我们可以取前七天的统计值作为特征, 也就是将 $t-1 \sim t-8$ 这个时间段数据的平均数、中位数、标准差、最大值、最小值等作为特征。
- window=1: 就是滞后值



序列特征提取案例

加载数据

```
import pandas as pd
import tushare as ts
import numpy as np
%matplotlib inline
# 通过tushare库获取股票代码
# 000001的历史收盘价数据
price = ts.get_hist_data("000001")[['close']] #只保留close变量
price.sort_index(inplace=True) #按照index升序排序, 默认是降序
price.head()
```

close	
date	
2019-03-18	12.91
2019-03-19	12.79
2019-03-20	12.75
2019-03-21	12.69
2019-03-22	12.59

序列特征提取案例

提取滞后值特征

```
# 把时间序列转换为机器学习数据格式先初始化一个空的dataframe名字叫df, 用来保存转换后的数据集
df = pd.DataFrame()
# 针对每一天, 我们取其过去10天 (不含当天) 的滞后值作为X变量, 把当天的明天值作为目标变量y
for i in range(10, 0, -1):
    df['t-' + str(i)] = price.close.shift(i)
# shift(i)函数的作用是取滞后值, 如果i=1就相当于昨天, i=2相当于前天...
df['t'] = price.close # 把当前时刻的值赋值给变量t
# 把当前时刻的明天值赋值给变量t+1--作为目标变量y
df['t+1'] = price.close.shift(-1)
# 查看前11条记录
df.head(11)
```

	t-10	t-9	t-8	t-7	t-6	t-5	t-4	t-3	t-2	t-1	t	t+1
date												
2019-03-18	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	12.91	12.79
2019-03-19	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	12.91	12.79	12.75
2019-03-20	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	12.91	12.79	12.75	12.69
2019-03-21	NaN	NaN	NaN	NaN	NaN	NaN	NaN	12.91	12.79	12.75	12.69	12.59
2019-03-22	NaN	NaN	NaN	NaN	NaN	NaN	12.91	12.79	12.75	12.69	12.59	12.11
2019-03-25	NaN	NaN	NaN	NaN	NaN	12.91	12.79	12.75	12.69	12.59	12.11	12.10
2019-03-26	NaN	NaN	NaN	NaN	12.91	12.79	12.75	12.69	12.59	12.11	12.10	12.38
2019-03-27	NaN	NaN	NaN	12.91	12.79	12.75	12.69	12.59	12.11	12.10	12.38	12.22
2019-03-28	NaN	NaN	12.91	12.79	12.75	12.69	12.59	12.11	12.10	12.38	12.22	12.82
2019-03-29	NaN	12.91	12.79	12.75	12.69	12.59	12.11	12.10	12.38	12.22	12.82	13.18
2019-04-01	12.91	12.79	12.75	12.69	12.59	12.11	12.10	12.38	12.22	12.82	13.18	13.36

序列特征提取案例

特征组合

'''

另外，由于数据集中前10条数据的特征变量不完整（因为我们要取过去10天的滞后数据，它们是最早的10条数据，所以会导致某些天的滞后数据缺失），所以我们在建模的时候会把前10条数据删除掉。最新日期的一条数据的t+1变量的值也会为空，我们也删除掉它。

'''

删除掉缺失数值

df = df.dropna()

只取前11个特征变量

df_X = df.iloc[:, :11]

df_X['mean'] = df_X.apply(np.mean, axis=1) #过去11天均值

df_X['min'] = df_X.apply(np.min, axis=1) #过去11天最小值

df_X['max'] = df_X.apply(np.max, axis=1) #过去11天最大值

df_X['t_div_mean'] = df_X['t']/df_X['mean'] #当天值/过去11天均值

df_X['t_div_min'] = df_X['t']/df_X['min'] #当天值/过去11天最小值

df_X['t_div_max'] = df_X['t']/df_X['max'] #当天值/过去11天最大值

df_X.head()

mean	min	max	t_div_mean	t_div_min	t_div_max
12.594545	12.1	13.18	1.046485	1.089256	1.0
12.635455	12.1	13.36	1.057342	1.104132	1.0
12.694545	12.1	13.44	1.058722	1.110744	1.0
12.795455	12.1	13.86	1.083197	1.145455	1.0
12.910909	12.1	13.96	1.081256	1.153719	1.0

计算机如何理解文本？

我们来看下面的两段话计算机是如何处理的：

小明喜欢看电影，小红也喜欢看电影。

小明还喜欢看足球比赛。

直接看结果：

小明/喜欢/看/电影/， /小红/也/喜欢/看/电影/。
小明/还/喜欢/看/足球比赛/。



小明	喜欢	看	电影	小红	也	还	足球比赛
1	2	2	2	1	1	0	0
1	1	1	0	0	0	1	1

相关概念

- 每一个文本中的词都被切开了，我们把这个叫**分词**或者叫 **tokenize**
- 每一个文本都用一组词对应的数值来表示它，这就变成**结构化数据**了。每个词对应的数值就是该词在文本中出现的次数。
- 上述的这种对文本的特征表示方法就称为“**词袋**”模型，词袋的英文是 **Bag of Words**，所以有时候又把词袋叫 **BOW**

小明/喜欢/看/电影/， /小红/也/喜欢/看/电影/。
小明/还/喜欢/看/足球比赛/。



小明	喜欢	看	电影	小红	也	还	足球比赛
1	2	2	2	1	1	0	0
1	1	1	0	0	0	1	1

词袋模型特点

词袋模型有这些的一些特点：

- 由于词典数量较多，而单一文本中出现的词并不会太多，所以用词袋模型表示的特征向量是**非常稀疏**的。
- 它**忽略了文本的语法和不同词之间的顺序**，仅仅将其看成是若干个词汇的集合，每个词的出现都是独立的不同词语的**重要性**仅仅是根据出现的**次数**来表示。
- 但真实的语言中可能并不是如此。
- 用**词频 (TF)** 来表示一个词在文本中的重要性有一个**致命的缺点**：常用词的重要性会被放大。因为在一段文本中可能常用词，例如“我们”会多次出现，但是它对文本的重要性其实并没有那么大，当然大多数情况下出现得多的词重要性也会相对较强，所以不能一概而论。

TFIDF

那么应该如何平衡这个问题呢？那就是TF-IDF算法，先来看几个术语：

- **TF (term frequency)**：词频，某个词在文档中出现的次数，TF 越大一般来说越重要
- **DF (document frequency)**：文档频率，某个词在所有文档中出现的文档数，DF 越大表示这个词越有可能是常用词，自然也越不重要
- **IDF (inverse document frequency)**：逆文档频率，它是 DF 的倒数，IDF 越大表示该词越少见，也越重要
- **TF-IDF**： $TF * IDF$ ，综合了 TF 和 IDF 两个因素来平衡词的重要性

TFIDF 计算流程

- 小明/喜欢/看/电影/, /小红/也/喜欢/看/电影/。
- 小明/还/喜欢/看/足球比赛/。

词语IDF

词语	DF	IDF
小明	2	1
喜欢	2	1
看	2	1
电影	1	1.4
小红	1	1.4
也	1	1.4
还	1	1.4
足球比赛	1	1.4

TF (归一化)

小明	喜欢	看	电影	小红	也	还	足球比赛
0.11	0.22	0.22	0.22	0.11	0.11	0	0
0.2	0.2	0.2	0	0	0	0.2	0.2



TF-IDF

小明	喜欢	看	电影	小红	也	还	足球比赛
0.11	0.22	0.22	0.31	0.16	0.16	0	0
0.2	0.2	0.2	0	0	0	0.28	0.28



词表征

词表征就是如何用向量的方式来表示一个词的特征，让计算机能够对词进行处理，常用的两种词表征的方法：

- **词袋模型**：一个词也可以理解为是一篇最简单的文档，所以它可以用词袋来表征它的特征，这个时候的词袋就是一个**独热编码**。
- **词向量模型**：这就是我们本节要讲解的方法，后面介绍。

词表征：词袋模型

每个词表示成一个one-hot向量

onehot

小明/喜欢/看/电影/, /小红/也/喜欢/看/电影/。
小明/还/喜欢/看/足球比赛/。



小明	喜欢	看	电影	小红	也	还	足球比赛
1	2	2	2	1	1	0	0
1	1	1	0	0	0	1	1

小明	1	0	0	0	0	0	0	0
喜欢	0	1	0	0	0	0	0	0
看	0	0	1	0	0	0	0	0
电影	0	0	0	1	0	0	0	0
小红	0	0	0	0	1	0	0	0
也	0	0	0	0	0	1	0	0
还	0	0	0	0	0	0	1	0
足球比赛	0	0	0	0	0	0	0	1

词表征：词袋模型

小明/喜欢/看/电影/, /小红/也/喜欢/看/电影/。
小明/还/喜欢/看/足球比赛/。



小明	喜欢	看	电影	小红	也	还	足球比赛
1	2	2	2	1	1	0	0
1	1	1	0	0	0	1	1

小明/喜欢/看/电影, 小红/也/喜欢/看/电影。

小明 1 0 0 0 0 0 0 0

喜欢 0 1 0 0 0 0 0 0

看 0 0 1 0 0 0 0 0

电影 0 0 0 1 0 0 0 0

小红 0 0 0 0 1 0 0 0

也 0 0 0 0 0 1 0 0

还 0 0 0 0 0 0 1 0

足球比赛 0 0 0 0 0 0 0 1

1 2 2 2 1 1 0 0

词向量

词向量，英文叫 **Word2Vec**，又叫词嵌入 (Word Embedding)，这种方法可以解决词袋模型的稀疏性问题，它的核心思想是：每一个词映射到一个多维空间中，成为空间中的一个向量，一般这个多维空间的维数不会太高，在几百个的量级。这几百维的特征向量是稠密的，向量中的每一个成员值都是非0的，例如：

“我” 这个词可以表征为：[0.4, -0.11, 0.55, 0.3 ... 0.1, 0.02]

“喜欢” 这个词可以表征为：[-0.02, -0.09, 0.04, 0.02 ..., 0.5, 0.03]

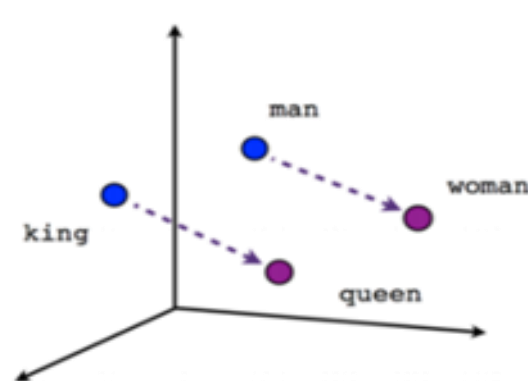
由于词向量由几百个维度构成，所以也被称为**分布式表征 (Distributed Representation)**。
词向量模型是通过对原始文本建模训练学习得到的。

词向量

由于词向量把每一个词映射到了一个高维空间中，并用向量表示，向量的生成是基于词与词之间的相关性得来，可以理解为相关的词在空间中的位置会比较靠近，所以词向量有一个非常有趣的特性，那就是类比。如下图所示，我们对不同词的词向量进行运算可以得到有趣的结果：

$\text{vector}(\text{"国王"}) - \text{vector}(\text{"王后"}) \approx \text{vector}(\text{"男人"}) - \text{vector}(\text{"女人"})$

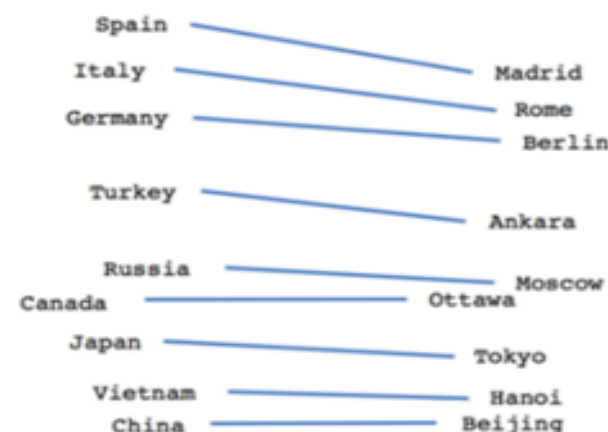
$\text{vector}(\text{"英国"}) + \text{vector}(\text{"首都"}) \approx \text{vector}(\text{"伦敦"})$



Male-Female



Verb tense



Country-Capital

Word2Vec 原理：基于邻居词共现矩阵分解法

下图示例了 $c=1$ 的情况下的词与词的共现矩阵：

所有文档 = {"I like deep learning"
"I like NLP"
"I enjoy flying"}

	I	like	enjoy	deep	learning	NLP	flying	.
I	0	2	1	0	0	0	0	0
like	2	0	0	1	0	1	0	0
enjoy	1	0	0	0	0	0	1	0
deep	0	1	0	0	1	0	0	0
learning	0	0	0	1	0	0	0	1
NLP	0	1	0	0	0	0	0	1
flying	0	0	1	0	0	0	0	1
.	0	0	0	0	1	1	1	0

共现次数

Word2Vec 原理：神经网络训练

通过构建两种类型的预测模型，然后使用网络的隐藏层输出作为词向量表征，这两种预测模型是：

- CBOW：利用中心词的邻居词预测中心词
- Skip-gram：利用中心词来预测邻居词

不管是哪种类型的神经网络，它的本质都是希望发现**中心词和邻居词之间的相关关系**，**词向量就是隐藏在这个相关关系中的隐特征**。

文档表征

词向量只是对词的特征表征，如果要对一篇文档进行特征表征，该如何做呢？有以下几种方法可以尝试：

- 直接使用文档中所有词的词向量的平均值
- 使用文档中每个词的TF-IDF值作为权重，与每个词的词向量进行加权平均
- 根据文档中每个词的词向量对文档进行聚类，使用聚类后包含词最多的那个类的中心点作为文档特征向量
- 使用 doc2vec 模型，这是个类似 word2vec 的模型，不过它是直接对 doc来建模

具体哪种方法好还要看你拿文档向量去解决的问题而定，所以这就需要具体问题具体评估来看了。

TFIDF 特征提取案例

导入需要使用的工具类

```
import jieba
from sklearn.feature_extraction.text import CountVectorizer, TfidfTransformer, TfidfVectorizer
import pandas as pd
```

```
contents = [
    '小明喜欢看电影，小红也喜欢看电影。',
    '小明还喜欢看足球比赛。'
]#就是我们的语料库
stopwords={'，','，','。'}
```

TFIDF 特征提取案例

使用 CounterVectorizer 提取TF词频特征

```
# 计算TF (每个词的出现次数, 未归一)
# tokenizer: 定义一个函数, 接受文本, 返回分词的list
# stop_words: 定义停用词词典, 会在结果中删除词典中包含的词
tf = CountVectorizer(tokenizer=jieba.lcut, stop_words=stopwords)
res1 = tf.fit_transform(contents) #contents中是两句话 结果就是两个向量
res1 #这是一个稀疏矩阵 稀疏矩阵保存数据更节省空间
```

```
<2x9 sparse matrix of type '<class 'numpy.int64''>'
  with 12 stored elements in Compressed Sparse Row format>
```

```
res1.toarray()
```

```
array([[1, 2, 1, 1, 2, 2, 0, 0, 1],
       [0, 1, 1, 0, 0, 1, 1, 1, 0]])
```

TFIDF 特征提取案例

使用 TfidfTransformer 计算TF-IDF特征

```
# use_idf: 表示在TF矩阵的基础上计算IDF, 并相乘得到TF-IDF
# smooth_idf: 表示计算IDF时, 分子上的总文档数+1
# sublinear_tf: 表示使用  $1+\log(tf)$  替换原来的tf
# norm: 表示对TF-IDF矩阵的每一行使用L2范数正则化(变成单位向量 模长为1) L1范数就是绝对值的和 max正则化---最大值
tfidf = TfidfTransformer(norm='l2', use_idf=True, smooth_idf=True, sublinear_tf=False)
res2=tfidf.fit_transform(res1) #res1是前面tf转换的结果
res2.toarray() #得到的就是tfidf向量
```

```
array([[0.29416605, 0.41860314, 0.20930157, 0.29416605, 0.58833211,
        0.41860314, 0.          , 0.          , 0.29416605],
       [0.          , 0.37930349, 0.37930349, 0.          , 0.          ,
        0.37930349, 0.53309782, 0.53309782, 0.          ]])
```

```
tf.vocabulary_.items()
```

```
dict_items([('小明', 2), ('喜欢', 1), ('看', 5), ('电影', 4), (' ', 8), ('小红', 3), ('也', 0), ('还', 7), ('足球比赛', 6)])
```

```
# 查看每个词的IDF值
```

```
tfidf.idf_
```

```
array([1.40546511, 1.          , 1.          , 1.40546511, 1.40546511,
        1.          , 1.40546511, 1.40546511, 1.40546511])
```

TFIDF 特征提取案例

直接使用 TfidfVectorizer提取TF-IDF特征

```
# 参数为 CounterVectorizer 和 TfidfTransformer 的所有参数
tfidf=TfidfVectorizer(tokenizer=jieba.lcut, stop_words=stopwords, norm='l2', use_idf=True, smooth_idf=True, sublinear_tf=False)
res=tfidf.fit_transform(contents) #直接对文档进行转换提取tfidf特征
res.toarray() #一步就得到了tfidf向量
```

```
array([[0.29416605, 0.41860314, 0.20930157, 0.29416605, 0.58833211,
        0.41860314, 0.         , 0.         , 0.29416605],
       [0.         , 0.37930349, 0.37930349, 0.         , 0.         ,
        0.37930349, 0.53309782, 0.53309782, 0.         ]])
```

```
pd.DataFrame(res.toarray(), columns=[x[0] for x in sorted(tf.vocabulary_.items(), key=lambda x:x[1])])
```

	也	喜欢	小明	小红	电影	看	足球比赛	还	,
0	0.294166	0.418603	0.209302	0.294166	0.588332	0.418603	0.000000	0.000000	0.294166
1	0.000000	0.379303	0.379303	0.000000	0.000000	0.379303	0.533098	0.533098	0.000000

Word2Vec 特征提取案例

导入数据

```
import numpy as np
import pandas as pd
```

```
# 查看训练数据
```

```
train_data=pd.read_csv('data/sohu_train.txt', sep='\t', header=None, dtype=np.str_, encoding='utf8', names=['频道', '文章'])
train_data.info()
```

	频道	文章
0	娱乐	《青蛇》造型师默认新《红楼梦》额妆抄袭 (图) 凡是看过电影《青蛇》的人, 都不会忘记青白二蛇的...
1	娱乐	6. 16 日剧榜 <最后的朋友> 亮最后杀招成功登顶 《最后的朋友》本周的电视剧排行榜单依然只...
2	娱乐	超乎想象的好看《纳尼亚传奇 2: 凯斯宾王子》 现时资讯如此发达, 搜狐电影评审团几乎人人在没有看...
3	娱乐	吴宇森: 赤壁大战不会出现在上集 "希望《赤壁》能给你们不一样的感觉。" 对于自己刚刚拍完的影片...
4	娱乐	组图: 《多情女人痴情男》陈浩民现场耍宝 陈浩民: 外面的朋友大家好, 现在是搜狐现场直播, 欢迎《...

Word2Vec特征提取案例

创建输出目录，加载停用词

```
# 创建输出目录 用来保存训练好的词向量
output_dir='output_word2vec'
import os
if not os.path.exists(output_dir):
    os.mkdir(output_dir)
```

```
# 载入停用词
stopwords = set()
with open('data/stopwords.txt', 'r', encoding='utf8') as infile:
    for line in infile:
        line = line.rstrip('\n')
        if line:
            stopwords.add(line.lower())
```


Word2Vec特征提取案例

使用 jieba 对文档进行分词

```
# 分词
import jieba
article_words=[]
# 遍历每篇文章
for article in train_data[u'文章']:
    curr_words=[]
    # 遍历文章中的每个词
    for word in jieba.cut(article):
        # 去除停用词
        if word not in stopwords:
            curr_words.append(word)
    article_words.append(curr_words)
```

```
# 分词结果存储到文件
seg_word_file=os.path.join(output_dir, 'seg_words.txt')
with open(seg_word_file, 'wb') as outfile:
    for words in article_words:
        outfile.write(u' '.join(words).encode('utf8') + b'\n')
print('分词结果保存到文件: {}'.format(seg_word_file))
```

Word2Vec 特征提取案例

训练 word2vec 模型

```
from gensim.models import Word2Vec
from gensim.models.word2vec import LineSentence
# 创建一个句子迭代器，一行为一个句子，词和词之间用空格分开
# 这里我们把一篇文章当作一个句子
sentences=LineSentence(seg_word_file)
# 训练word2vec模型
# 参数说明:
# sentences: 包含句子的list, 或迭代器
# vector_size: 词向量的维数, size越大需要越多的训练数据, 同时能得到更好的模型
# alpha: 初始学习速率, 随着训练过程递减, 最后降到 min_alpha
# window: 上下文窗口大小, 即预测当前这个词的时候最多使用距离为window大小的词
# max_vocab_size: 词表大小, 如果实际词的数量超过了这个值, 过滤那些频率低的
# workers: 并行度
# epochs: 训练轮数
# min_count: 忽略出现次数小于该值的词
model=Word2Vec(sentences=sentences, vector_size =100, epochs=10, min_count=20)
# 保存模型
model_file = os.path.join(output_dir, 'model.w2v')
model.save(model_file)
```

Word2Vec特征提取案例

查找语义相近的词

```
: def invest_similar(*args, **kwargs):  
    res=model2.wv.most_similar(*args, **kwargs)  
    print('\n'.join([' {}: {}'.format(x[0],x[1]) for x in res]))
```

```
: invest_similar(u' 摄影', topn=5)
```

刘晓科:0.5886282920837402

刘建东:0.5798802971839905

作曲:0.5547745227813721

摄像:0.5399066209793091

杨文:0.5288978815078735

```
: # 女人 + 先生 - 男人 = 女士  
# 先生 - 女士 = 男人 - 女人, 这个向量的方向就代表了性别!  
invest_similar(positive=[u' 女人', u' 先生'], negative=[u' 男人'], topn=1)
```

女士:0.6695073246955872

Word2Vec特征提取案例

计算两个词的相似度

```
model2.wv.similarity('摄影', '摄像')
```

0.5399067

```
model2.wv[u' 摄影']
```

```
array([-3.1606889 , -0.5189758 , -2.1030898 , -0.03520009, -2.3420804 ,  
       -0.5640013 ,  3.6445358 , -0.8599415 , -1.2179868 , -0.16347185,  
        1.8404166 , -1.5374502 , -5.5010505 ,  1.3621913 , -0.999915 ,  
       -2.0083125 ,  0.3612812 , -3.1573272 , -0.39578527,  1.6425959 ,  
        1.4099249 , -0.41359398,  4.7726865 , -0.49909878,  4.5835385 ,  
        0.1876434 , -1.6853429 , -1.3888341 , -1.1844304 , -2.053083 ,  
       -1.3943285 , -0.9651567 , -2.0514753 , -0.07557752, -0.33762047,  
        1.5513391 , -0.04874396,  0.7174364 ,  0.88150567, -0.64206713,  
       -3.0687008 ,  0.79496425, -0.2760602 , -0.24640591, -0.1327867 ,  
       -0.39856085,  1.3950926 ,  1.2337818 ,  0.6685925 ,  0.13246498,  
       -0.0174366 ,  1.698126 , -0.22738287, -4.7165623 ,  0.33348256,  
       -0.9649006 , -0.7675093 ,  0.14599355, -0.06840796,  1.609182 ,  
        1.0826575 , -0.16944721,  1.1718382 , -1.0852969 ,  0.9098468 ,  
        1.18416 ,  0.07775807, -0.3061563 , -0.18089724,  0.15941624,  
        0.47926268,  0.602958 ,  0.84622353, -0.9514689 ,  0.20886804,  
       -1.8178835 , -3.4758341 , -0.4116545 ,  1.1929843 , -0.11298775,  
       -1.6553439 , -0.1522164 ,  2.3426704 , -2.1387005 , -0.5657439 ,  
        1.4761485 , -0.13485233, -1.1922555 ,  1.128761 , -1.1376222 ,  
       -0.98006606, -3.17027 , -4.2168345 ,  1.3973597 ,  0.31865987,  
        1.3277856 ,  1.4765654 , -0.01264292, -1.5258672 ,  2.3189952 ],  
      dtype=float32)
```

下面哪些可能是一个文本语料库的特征（）

1.一个文档中的词频统计

2.文档中单词的布尔特征

3.词向量

4.词性标记

5.基本语法依赖

6.整个文档

A. 123

B. 1234

C. 12345

D. 123456

1
3
chapter

特征组合

为什么要进行特征组合

通过特征组合可以构造出更多更好的特征，可以提升模型的效果。

- 举一个直观的例子：我们知道地球上每一个区域都有**经度**和**纬度**两个特征，如果把经度和纬度当做独立的两个特征，那么只能分析出经度或者纬度对目标的单一影响。但是如果把经纬度组合起来，我们就能对地球上任意组合的特定区域进行分析，得出**经纬度**的组合对目标的影响。在这个例子中，把**经纬度**组合起来作为一个新的特征，这就叫特征组合。

组合方式

针对不同类型的变量，特征组合的方式也不一样：

- 如果两个特征变量都是**类别型变量**，例如性别（男/女）和年龄段（1,2,3），我们把两个变量的类别值进行两两交叉组合就得到一个新的类别型变量，例如性别年龄组合后的结果就是：男-1，男-2，男-3，女-1，女-2，女-3，新的特征共有6个类别值。
- 如果两个特征变量都是**连续型变量**，我们可以通过两个变量的**四则运算**来组合出新特征，例如：
： $x_1 * x_2$ 或者 x_1 / x_2

多项式扩展

两维线性模型:

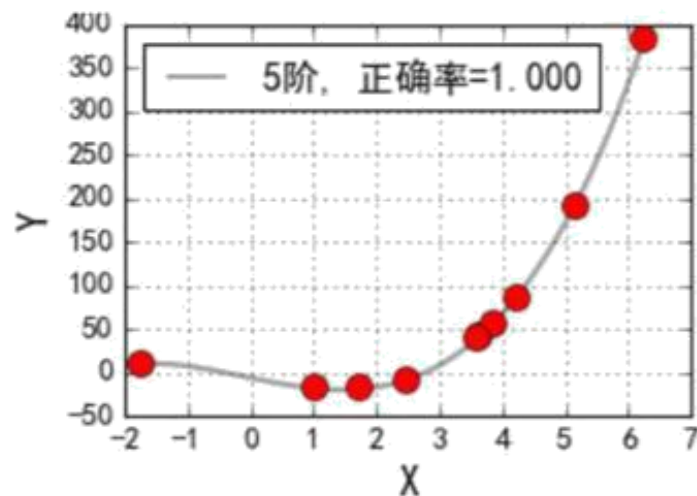
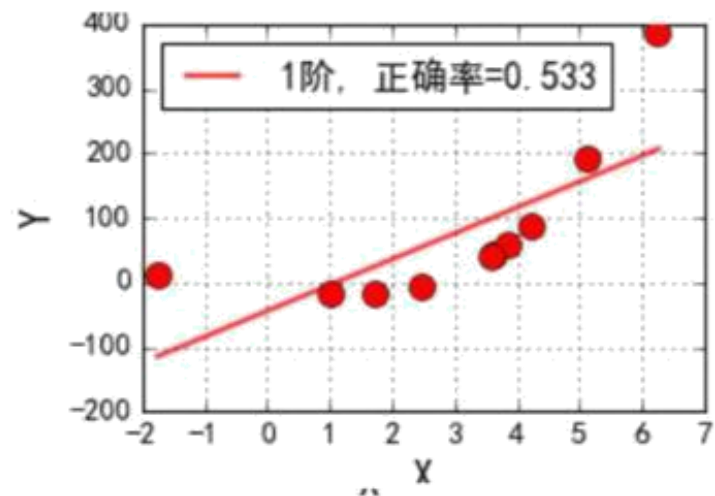
$$h_{\theta}(x_1, x_2) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$(x_1, x_2) \xrightarrow{\text{多项式扩展}} (x_1, x_2, x_1^2, x_2^2, x_1 x_2)$$

五维线性模型:

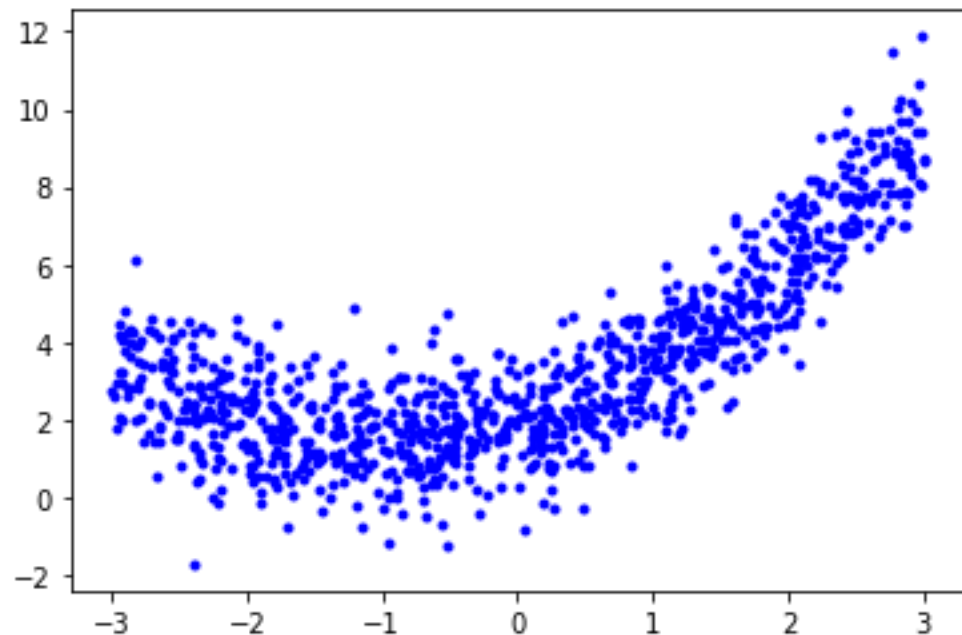
$$h_{\theta}(x_1, x_2) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_1^2 + \theta_4 x_2^2 + \theta_5 x_1 x_2$$

$$\xrightarrow{\text{等价}} h_{\theta}(x_1, x_2) = \theta_0 + \theta_1 z_1 + \theta_2 z_2 + \theta_3 z_3 + \theta_4 z_4 + \theta_5 z_5$$



1 构造数据集

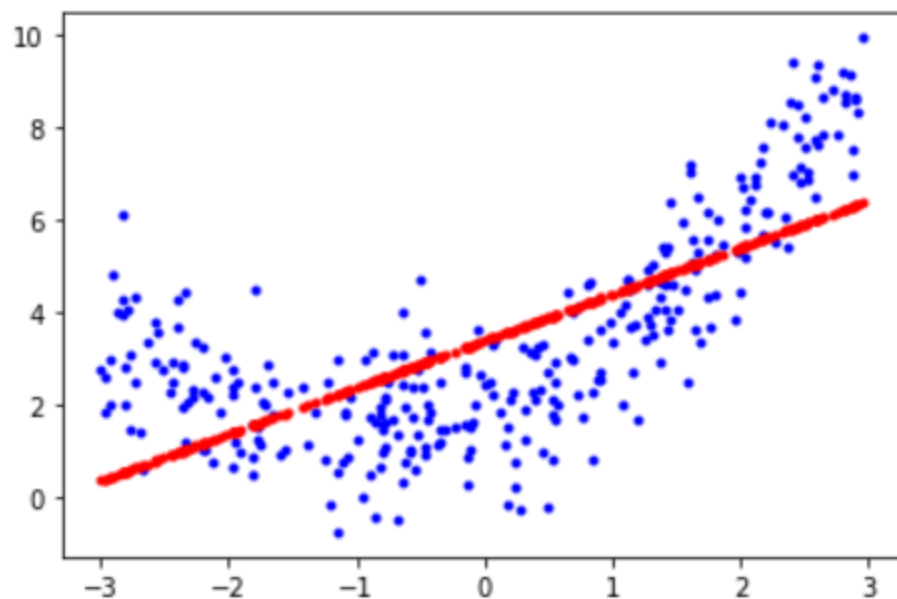
```
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline
# 构造x和y变量, 以满足二项式关系, 并增加一些随机误差
records = 1000
#构造数据集的记录数
X = 6 * np.random.rand(records, 1) - 3
y = 0.5 * X**2 + X + 2 + np.random.randn(records, 1)
# 绘图
plt.plot(X, y, 'b.')
plt.show()
```



2 直接进行线性回归建模

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
# 拆分数数据集和测试集
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=123456)
# 模型训练
lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)
# 模型评估
print("R2:", lin_reg.score(X_test, y_test))
# 绘制拟合线
y_pred = lin_reg.predict(X_test)
plt.plot(X_test, y_test, 'b.')
plt.plot(X_test, y_pred, 'r.', linewidth=3)
plt.show()
```

R2: 0.46230466926866043



3 多项式扩展后建模

```
from sklearn.preprocessing import PolynomialFeatures
# 实例化多项式特征转换器
poly_features = PolynomialFeatures(degree=2, include_bias=False)
# 转换数据得到多项式特征
X_poly = poly_features.fit_transform(X)
X_poly
```

```
array([[ 2.76942041,  7.66968941],
       [-0.7747418 ,  0.60022485],
       [ 0.65440543,  0.42824647],
       ...,
       [ 1.06444683,  1.13304706],
       [ 0.39107496,  0.15293963],
       [-0.6155175 ,  0.37886179]])
```

拆分数据集和测试集

```
X_train, X_test, y_train, y_test = train_test_split(X_poly, y, test_size=0.3, random_state=123456)
```

模型训练

```
lin_reg_poly = LinearRegression()
```

```
lin_reg_poly.fit(X_train, y_train)
```

模型评估

```
print("R2:", lin_reg_poly.score(X_test, y_test))
```

绘制拟合线

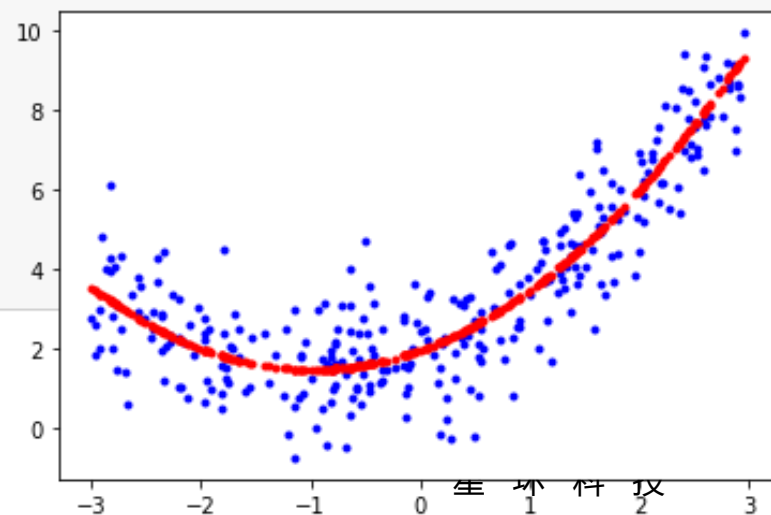
```
y_pred = lin_reg_poly.predict(X_test)
```

```
plt.plot(X_test[:, 0], y_test, 'b.')
```

```
plt.plot(X_test[:, 0], y_pred, 'r.')
```

```
plt.show()
```

R2: 0.801845752129297



FM模型

$$\text{FM: } \hat{y}(x) := w_0 + \underbrace{\sum_{i=1}^n w_i x_i}_{\text{LR模型}} + \underbrace{\sum_{i=1}^n \sum_{j=i+1}^n \langle v_i, v_j \rangle x_i x_j}_{\text{Dense化两两特征}}$$

LR模型

Dense化两两特征

$$\langle v_i, v_j \rangle := \sum_{f=1}^k v_{i,f} \cdot v_{j,f} = \begin{matrix} 0.1 & 0.2 & 0.6 & 0.8 & 0.1 & 0.2 \\ \hline 0.4 & 0.2 & 0.4 & 0.2 & 0.4 & 0.2 \end{matrix} \quad v_j$$

v_i

FM模型

$$\langle v_i, v_j \rangle := \sum_{f=1}^k v_{i,f} \cdot v_{j,f} = \begin{matrix} 0.1 & 0.2 & 0.6 & 0.8 & 0.1 & 0.2 \\ \hline 0.4 & 0.2 & 0.4 & 0.2 & 0.4 & 0.2 \end{matrix} \quad v_j$$

v_i

v_1	0.3	0.2	0.6	0.8	0.1	0.2
v_2	0.1	0.8	0.6	0.8	0.4	0.6
v_3	0.4	0.2	0.7	0.2	0.1	0.2
v_4	0.1	0.2	0.6	0.8	0.5	0.2
v_{n-1}	0.3	0.2	0.6	0.8	0.1	0.2
v_n	0.5	0.8	0.9	0.8	0.4	0.6



$w_{i,j} = \langle v_i, v_j \rangle \neq 0$
 even if 训练数据中 $x_i x_j = 0$
 only if 在训练数据中存在k使得 $x_i x_k \neq 0$

FM模型泛化能力强

1
4
chapter

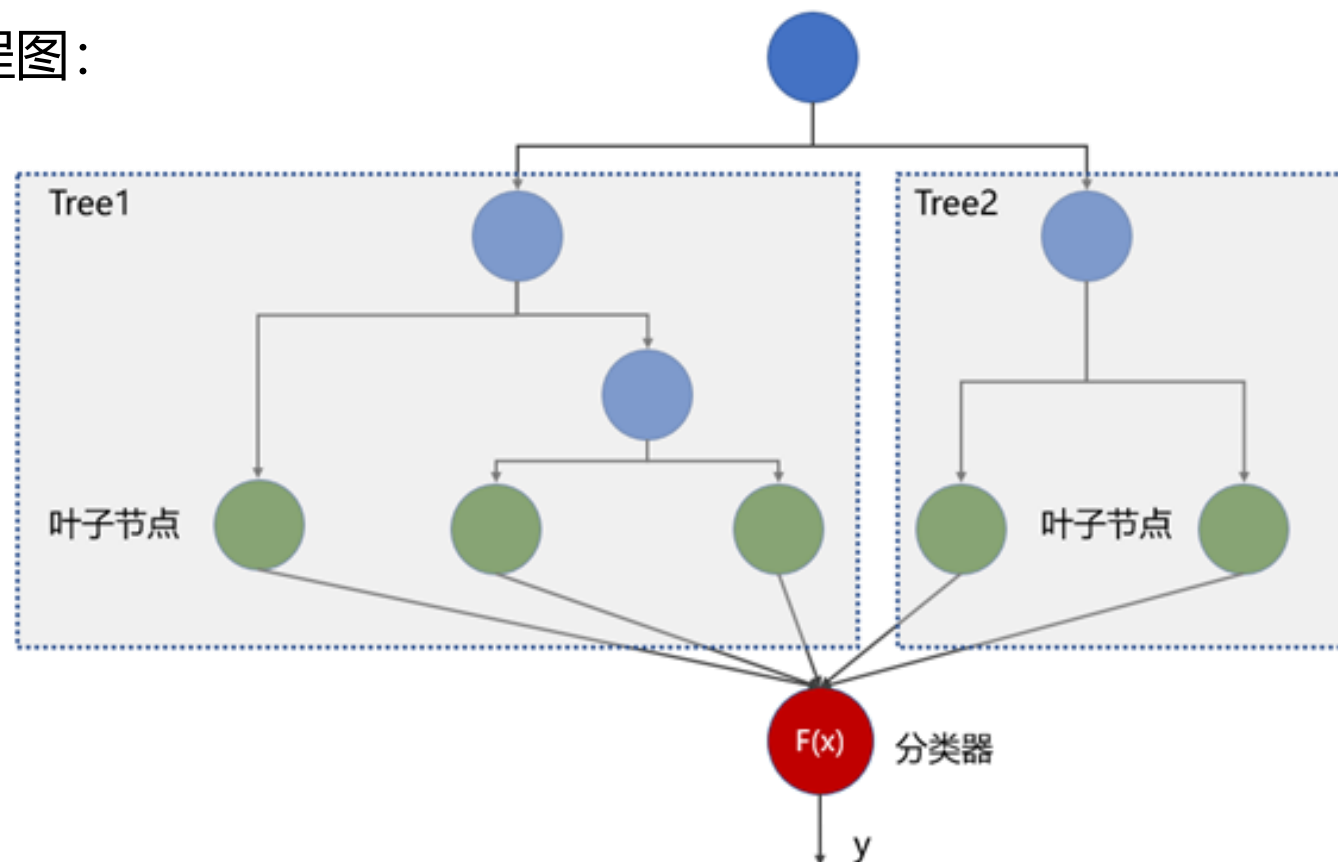
GBDT 特征提取

为什么要使用 GBDT 提取特征

为什么要选择 GBDT 来提取组合特征而不是选择普通的决策树模型？

■ 主要的原因是基于融合算法的树模型（例如随机森林和 GBDT 等）效果比普通决策树更好。

GBDT 提取特征流程图：



GBDT提取特征流程

- 先构建 GBDT 模型，得到若干个决策树子模型
- 提取样本在每一棵决策树上的组合特征：样本对应叶子节点ID的独热编码。
- 如果 GBDT 有 30 个决策树子模型，就提取 30 个这样的独热编码。提取完成之后得到样本的特征向量，类似这样的表示：(0,1,0,0...1,0,0,0....0,0,1,0)，特征向量的长度就是所有子模型的叶子节点总数，向量中取值为1的个数就是子模型的个数（因为一个样本在1个子模型中有且只能归属一个叶子节点）。

基于提取的特征向量构建预测模型，一般这个时候使用简单的线性模型（例如逻辑回归）效果就会非常好了，因为 GBDT 提取的组合特征已经拥有非常强的预测能力了。

准备数据

```
# 读取数据
titanic_df = pd.read_csv('./data/titanic/train.csv')
# drop掉PassengerId, Name, Ticket, Cabin
titanic_df = titanic_df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis = 1)
# 构造一个新变量AgeIsMissing
titanic_df['AgeIsMissing'] = 0
titanic_df.loc[titanic_df['Age'].isnull(), 'AgeIsMissing'] = 1
# 对Age缺失值进行均值填充
age_mean = round(titanic_df['Age'].mean())
titanic_df['Age'].fillna(age_mean, inplace=True)
# 对Embarked缺失值用'S'替换
titanic_df['Embarked'].fillna('S', inplace=True)
# 对Age进行分箱--自定义分箱
cut_points = [0, 18, 25, 40, 60, 100]
titanic_df["AgeBin"] = pd.cut(titanic_df.Age, bins=cut_points)
# 对Fare船票价格进行分箱--等深分箱
titanic_df["FareBin"] = pd.qcut(titanic_df.Fare, 5)
# 构造FamilySize变量
titanic_df['FamilySize'] = titanic_df['SibSp'] + titanic_df['Parch'] + 1
# 构造一个新变量IsAlone (是否独自一人)
titanic_df['IsAlone'] = 0
titanic_df.loc[titanic_df['FamilySize'] == 1, 'IsAlone'] = 1
# 构造一个新变量IsMother (是否是母亲)
titanic_df['IsMother'] = 0
titanic_df.loc[(titanic_df['Sex'] == 'female') & (titanic_df['Parch'] > 0) & (titanic_df['Age'] > 20), 'IsMother'] = 1
# 把Sex性别和AgeBin特征进行组合
titanic_df['SexAgeCombo'] = titanic_df['Sex'] + "_" + titanic_df['AgeBin'].astype(str)
# 对Pclass, Sex, Embarked, AgeBin, FareBin, FamilySize, Sex_Age_combo进行独热编码
Pclass = pd.get_dummies(titanic_df.Pclass, prefix='Pclass')
Sex = pd.get_dummies(titanic_df.Sex, prefix='Sex')
Embarked = pd.get_dummies(titanic_df.Embarked, prefix='Embarked')
AgeBin = pd.get_dummies(titanic_df.AgeBin, prefix='AgeBin')
FareBin = pd.get_dummies(titanic_df.FareBin, prefix='FareBin')
FamilySize = pd.get_dummies(titanic_df.FamilySize, prefix='FamilySize')
SexAgeCombo = pd.get_dummies(titanic_df.SexAgeCombo, prefix='SexAgeCombo')
```

准备数据

```
# 把需要的变量全部拼接在一起
TrainData = pd.concat([titanic_df[['Survived', 'AgeIsMissing', 'IsAlone', 'IsMother']],
                       Pclass, Sex, Embarked, AgeBin, FareBin, FamilySize, SexAgeCombo], axis=1)
# 准备数据集X和y, 并把他们转为numpy的array格式--Hyperopt只接受array格式的输入数据
trainData_X = TrainData.drop(['Survived'], axis = 1)
trainData_y = TrainData.Survived
trainData_X = trainData_X.values.astype(float)
trainData_y = trainData_y.values.astype(float)
# 拆分训练集和测试集
X_train, X_test, y_train, y_test = train_test_split(trainData_X, trainData_y, test_size=0.3, random_state=123456)
```

GBDT提取组合特征

Step2: GBDT提取组合特征

训练GBDT模型

#200个子模型，每棵决策树最大深度6

```
gbdt = GradientBoostingClassifier(n_estimators=200,max_depth=6)
```

```
gbdt.fit(X_train, y_train)
```

提取样本所属的叶子节点编码的特征（后续还要进行one-hot独热编码）

```
feature = gbdt.apply(X_train)
```

查看一下feature数据

```
print(feature.shape) #shape=(样本数, 子模型树, 1)
```

只取shape=(样本数, 子模型树)的数据

```
feature = feature[:, :, 0]
```

```
feature
```

```
(623, 200, 1)
```

```
array([[36.,  6., 35., ...,  7., 56.,  7.],  
       [24., 67., 24., ...,  6., 37., 69.],  
       [41., 12., 41., ...,  6., 56.,  6.],  
       ...,  
       [ 5., 61., 18., ..., 31.,  6., 39.],  
       [53., 27., 56., ...,  6., 81., 26.],  
       [50., 23., 51., ..., 16., 80., 26.]])
```

one-hot独热编码

```
# Step3: one-hot独热编码
enc = OneHotEncoder()
X_train_feature = enc.fit_transform(feature)
print(X_train_feature.shape)
X_train_feature.toarray()
```

```
(623, 8053)
array([[0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       ...,
       [0., 1., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.],
       [0., 0., 0., ..., 0., 0., 0.]])
```

```
# Step4:对X_test提取特征
feature = gbdt.apply(X_test)
feature = feature[:, :, 0]
X_test_feature = enc.transform(feature)
# 查看X_test_feature的shape
print(X_test_feature.shape)
```

```
(268, 8053)
```

逻辑回归训练模型和评估

```
# Step5: 用逻辑回归训练模型和评估
# 我们在新的训练数据集X_train_feature和X_test_feature上训练逻辑回归模型
# 用(X_train_feature, y_train)训练模型
lr = LogisticRegression()
lr.fit(X_train_feature, y_train)
# 评估测试集的accuracy
acc = lr.score(X_test_feature, y_test)
print("acc:", acc)
# 评估测试集的auc
y_pred = lr.predict_proba(X_test_feature)
fpr, tpr, thresholds = roc_curve(y_test, y_pred[:, 1])
auc_value = auc(fpr, tpr)
print("auc:", auc_value)
```

acc: 0.7835820895522388

auc: 0.7994330262225372



Q&A

TRANSWARP
星环科技